

A Bounded Reproducibility Package for the HSIT Program: Corrected Claims, Finite Regressions, and Open Controls

Author: Micah Blumberg

Self Aware Networks Research Institute

<https://selfawarenetworks.com>

July 28, 2026

Abstract

This revision corrects and narrows the reproducibility claims of an earlier methodology paper concerning a finite weighted Farey-Laplacian component of the H_SIT program. The historical public record contains a PDF but not the complete executable object described by that paper. We therefore stage a private, self-contained executable replay for the stated finite checks, with source, tests, pinned package versions, generated JSON, provenance, license disposition, review records, and an exact-scope SHA-256 manifest. The external mathematical proof sources are identified but are not contained or verified by the executable replay.

The bundle evaluates a one-dimensional kernel, a finite Mertens trace, weighted-degree and operator-norm bounds, and a finite Commutator-Pythagoras identity at Farey levels 8, 16, 32, and 64. Across pseudoinverse cutoffs $1e-8$, $1e-10$, $1e-12$, and $1e-14$, the largest absolute identity residual is $2.0641e-12$. These rows are numerical regressions against proof-owned statements, not proofs. A fixed-seed, unfitted magnetic-operator baseline at level 64 retains 820 bulk modes and 819 nearest-neighbor spacings, has Kolmogorov-Smirnov distance 0.727395592237 from the GUE Wigner surmise, and produces rounded-eigenvalue hash prefix 308fe8684567ce7b. This is one descriptive finite configuration, not an ensemble law or zeta-zero result.

The primary logical correction concerns an infinitely-often deficit floor. Finite all-below-floor observations cannot contradict a subsequence theorem unless a separate proof establishes floor-attainment coverage for the exact evaluated set, epsilon, and constant. Because a JSON object cannot verify such a proof, the code never emits or authorizes THEOREM-CONTRADICTION. Schema-complete coverage metadata can produce only CONDITIONAL-INCONSISTENCY, pending independent proof and exact-numerical review.

The staged artifact includes a fresh disposable-environment reproduction. CPython 3.12.10, NumPy 2.4.6, threadpoolctl 3.6.0, and OpenBLAS 0.3.31.188.0 produced two byte-identical result files with SHA-256 9f48dadbe876dc29ae8cc64c646b49641d3ce81024f743098adf6bb4ceeebfd1; all 22 tests passed, including three source-identity and claim-anchor tests. This establishes a second local environment build on the same Windows host, not an independent laboratory reproduction or a cross-platform guarantee. The zero-data analysis is permanently omitted from this revision. Holdout, fake-target, budget-equivalence, and all-runs controls remain unimplemented, and no trained result is reported. Nothing in this artifact proves the Riemann Hypothesis or physical Super Information Theory.

Scope and evidence boundary

selfawarenetworks.com

Prior public version: Zenodo record 21231891, DOI 10.5281/zenodo.21231891 (Blumberg 2026a)

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Code license: MIT **Scope:** The package demonstrates bounded finite-computation reproducibility; it does not prove RH, physical SIT, trained-engine performance, or cross-platform replication.

1. Revision Objective and Claim Architecture

Reproducibility belongs to an available research object under a bounded claim. It is not conferred by a protocol description alone. A deterministic program may reproduce its own output without reproducing a historical data-dependent result. A numerical regression may catch implementation drift without proving the theorem whose formula it evaluates. A complete software object may still be statistically inadequate for a scientific conclusion. These distinctions motivate the correction of the predecessor record (Blumberg 2026a).

The historical Paper 8 separated finite theorem checks, an unfitted operator baseline, and a proposed trained-engine protocol. Its public payload did not co-deposit the full source, environment, input data, raw output, tests, and hash inventory required by its strongest artifact claim. This revision preserves that historical fact. It does not retroactively describe the PDF-only release as executable. Instead, it stages a new private object for review.

The revised package uses six mutually exclusive evidence states:

1. **Proof-owned external statement:** a mathematical claim assigned to an identified companion source. The executable bundle does not independently establish its proof status.
2. **Regression:** a finite computation expected to agree with an exact or proof-owned statement and useful for detecting implementation drift.
3. **Descriptive:** a statistic measured under one explicit finite configuration, without asymptotic, ensemble, causal, or target-specific promotion.
4. **Synthetic fixture:** generated input used only to test software semantics.
5. **Permanently omitted from this revision:** an analysis deliberately removed from the revision's claim surface because its admissible input and provenance chain are absent.
6. **Unimplemented:** a required control for which no executable mechanism and no admissible result exist.

The generated field `overall: PASS` is limited to bounded replay and software contract checks. It does not mean that external proofs were validated, that a statistical model was accepted, that the manuscript is publication-ready, or that release is authorized. This scope is machine-readable in `overall_scope` and is protected by tests.

1.1 The problem and operation before the reproducibility label

Suppose a reader receives a paper that says a computation is reproducible. What, exactly, must be capable of happening again? A narrative can be read again without its calculations being rerun. A program can emit the same file again while depending on an unavailable historical dataset. A finite calculation can agree with a theorem-derived formula without checking the proof. A fixed random seed can reproduce one spectrum without establishing a population law. These are different achievements and they support different claims.

The established reproducibility account therefore asks for an inspectable research object: identified source, inputs, dependency versions, execution instructions, outputs, tolerances, provenance, and tests. Statistical claims add further requirements, including sampling definitions, controls, uncertainty, and a record of every run used to select the reported result. Mathematical claims additionally require proof review; numerical agreement is not a substitute.

The operation implemented here is deliberately narrower. The package:

1. reconstructs a finite weighted Farey object from declared rules;
2. evaluates identities and bounds at four declared finite levels;
3. records tolerances and runtime information;

4. generates a machine-readable result with bounded evidence classes;
5. rejects any attempt to turn missing theorem coverage into a contradiction;
6. preserves omitted data and unimplemented controls as explicit null states;
7. hashes the governed artifact set so later changes become visible.

Only after those actions are specified do we call the result a **bounded reproducibility package**. “Bounded” is essential: the package can establish that its finite implementation and claim guards replay under the declared environment. It cannot, by that fact alone, establish the external proofs, asymptotic behavior, a trained model result, RH, physical SIT, or independent cross-platform replication.

The proposal fails its own stated purpose if a fresh governed run cannot reproduce the finite rows within tolerance, if result classes can be silently promoted, if omitted inputs reappear without provenance, if a malformed coverage object can authorize a theorem verdict, or if the manifest does not detect a release-file mutation.

1.2 Evidence-decision matrix

The package is designed to prevent a successful check in one evidence class from being silently promoted into a stronger class.

Evidence object	What the package establishes	What it cannot establish
finite identity and bound regressions	the current implementation agrees with the stated finite formulas within declared tolerances	an independent proof, an infinite-volume theorem, RH, or physical SIT
deterministic repeated output	the same governed source and pinned runtime reproduce the same recorded bytes	portability, independent replication, or correctness of the scientific interpretation
fresh same-host environment	the package can be rebuilt locally without reusing the prior virtual environment	a materially independent investigator, operating system, architecture, or numerical backend
theorem-source hash and anchor contract	three current private source files match their recorded hashes and four named line ranges contain the required labels	proof validity, rights to redistribute, public reviewability, or independent mathematical endorsement
one-level, one-seed spacing discrepancy	the specified finite operator configuration yields the recorded descriptive distances	an unfolded spectral law, an ensemble result, a zeta-zero match, or a universality class
synthetic certificate fixtures	malformed and bounded diagnostic states obey the declared software semantics	a real floor-attainment proof or a theorem contradiction

A future release decision must therefore ask which row supports each sentence, not merely whether the package reports PASS.

2. Finite Mathematical Object

For an integer $N \geq 1$, let F_N be the closed-interval Farey sequence of reduced fractions in $[0, 1]$ with denominator at most N . Vertices p/q and r/s are connected when

$$|ps - rq| = 1. \quad [1]$$

The edge weight used by this artifact is

$$w(p/q, r/s) = 1/(qs)^2. \quad [2]$$

At zero magnetic phase, the finite weighted Laplacian L_N has diagonal entry equal to weighted degree and off-diagonal entry $-w(u,v)$ on each edge. It is a real symmetric positive-semidefinite matrix. The finite graph is connected, so the normalized constant vector

$$\phi_0 = |F_N|^{-1/2} \mathbf{1} \quad [3]$$

spans its kernel. Define the diagonal position unitary

$$(U \psi)(\rho) = \exp(2 \pi i \rho) \psi(\rho) \quad [4]$$

and let L_N^+ denote the Moore-Penrose pseudoinverse. The finite model and its proof obligations are owned by the weighted-Farey companion manuscript and the exact source packets identified by hash in reproducibility/PROVENANCE.md (Blumberg 2026c).

2.1 Finite Mertens Trace

Grouping the diagonal expectation by denominator uses the Ramanujan sum $c_q(1)=\mu(q)$. Under the closed-interval convention, 0/1 contributes an additional endpoint term. Thus

$$\langle \phi_0, U \phi_0 \rangle = (M(N)+1)/|F_N|, \quad [5]$$

where $M(N)=\sum_{q \leq N} \mu(q)$. The replay computes $\mu(1), \dots, \mu(N)$ with a deterministic linear sieve and compares the complex floating-point expectation with the rational formula under a declared tolerance. That comparison checks the implementation; it does not establish a new result about the Mertens function.

2.2 Finite Commutator-Pythagoras Identity

For the finite model,

$$|\langle \phi_0, U \phi_0 \rangle|^2 + \|L_N^+[U, L_N] \phi_0\|^2 = 1. \quad [6]$$

The proof-owned argument is finite spectral decomposition. Since $L_N \phi_0=0$,

$$[U, L_N] \phi_0 = -L_N U \phi_0. \quad [7]$$

The commutator vector is orthogonal to the kernel. Applying L_N^+ recovers the positive-spectrum projection of $-U \phi_0$; Parseval then separates the unit norm into kernel and positive-spectrum components. The code checks kernel dimension and commutator orthogonality before evaluating the identity. Passing output is a regression against this argument, not an independent proof audit.

2.3 Weighted Operator Bound

The identified companion source gives, for a vertex p/q ,

$$W(p/q) \leq \pi^2/(3q^2). \quad [8]$$

A row-sum or quadratic-form estimate then yields

$$\|L_N\| \leq 2 \pi^2/3. \quad [9]$$

The package checks the finite per-vertex and global inequalities. The owning proof source is named and hashed in the provenance record but is not redistributed here. A replay therefore confirms agreement of this implementation with the inequality. It cannot independently certify the source proof from the executable package alone.

2.4 Exact Deficit Identity and RH Boundary

Define

$$\delta(N) = 1 - ||L_N^{+}[U, L_N] \phi_0||^2. \quad [10]$$

The finite identity gives

$$\delta(N) = ((M(N)+1)/|F_N|)^2. \quad [11]$$

The companion saturation manuscript relates this identity to classical Mertens-function formulations of RH (Blumberg 2026b). In the notation used here, the upper-bound target is

$$\delta(N) = O(N^{-3+\epsilon}) \quad [12]$$

for every positive epsilon, after the usual rescaling of epsilon constants. This is a reformulation target, not a proof of RH. Classical primary work on zeta zeros and Mertens-type oscillation supplies historical context but does not promote the computational rows into theorem evidence (Hardy 1914; Ingham 1942).

The external sharpness statement that controls the certificate boundary has a different quantifier: for every $\epsilon > 0$, there exists $c(\epsilon) > 0$ such that

$$\delta(N) \geq c(\epsilon) N^{-3-\epsilon} \quad [13]$$

for infinitely many N . Version 2 continues to call this a proof-owned external statement. Neither the generated JSON nor its unit tests independently verify that proof.

2.5 Theorem-source identity contract

The private source contract records three proof packets and four exact claim anchors: Theorems C-4 and C-5 in BC-001, Theorem C-22 in BC-009, and Theorem C-25 in BC-012. The verifier recomputes each file hash and checks required text inside the declared line range. The generated verification record contains no machine-local path and currently reports PASS.

This is an identity and routing result. It establishes that the executable regressions point to the intended current private source bytes. It does not validate the proofs, establish that every assumption transfers to the implementation, authorize packet redistribution, or supply a stable public locator. Those are four separate gates: source identity, proof review, rights, and public reviewability. Version 2 keeps all four statuses explicit so that a content hash cannot be mistaken for mathematical review.

3. Coverage-Safe Certificate Semantics

Fix epsilon and a positive working constant c . Let S be the exact finite set of evaluated levels. Observing

$$\delta(N) < c N^{-3-\epsilon} \text{ for every } N \text{ in } S$$

does not contradict an infinitely-often lower bound. The theorem may be satisfied only at levels outside S . Increasing the size of S , observing a steep fitted slope, or encountering an exact zero does not repair the missing quantifier. A finite set becomes theorem-relevant only if a separate argument proves that S contains a floor-attaining level for the same epsilon and constant.

3.1 Coverage Metadata Is Not Proof Verification

The evaluator accepts an optional coverage object containing:

- the premise type evaluated-level-set-contains-floor-attainer;

- a claimed proof status;
- independence from the measurements;
- a nonempty proof reference;
- the exact evaluated levels;
- the same epsilon and working constant.

The code validates this schema and exact field agreement. It does not parse a proof, resolve the proof reference, check assumptions, verify exact arithmetic, or establish that the proof applies to the measured operator. The object is self-attested metadata. Even a schema-complete object can authorize only `CONDITIONAL-INCONSISTENCY`; `theorem_contradiction_authorized` remains false.

This design prevents a label such as `proof_status:\proved` from becoming a machine authority. A human or formal proof review must independently validate the referenced theorem, its constants, its level-set statement, and the exact comparison before any contradiction language is considered.

3.2 Bounded Diagnostic Rule

Without valid coverage metadata, the evaluator returns one of three bounded states:

- `INCONCLUSIVE` for malformed, nonfinite, negative, nonincreasing, or insufficient inputs;
- `NO-BOUNDED-FLAG` when the finite warning rule does not trigger;
- `BOUNDED-DIAGNOSTIC` when the evaluated tail lies below the working envelope and either the fitted positive-deficit slope is steeper than -3-epsilon or the tail contains a zero.

These states are software diagnostics, not asymptotic pass / fail verdicts. The comparison is float64 and strict. Exact theorem adjudication would require exact or interval arithmetic plus a validated coverage theorem, neither of which is supplied.

4. Reproducibility Method

4.1 Pinned Versions and Runtime Fingerprint

The generator requires CPython 3.12.10, NumPy 2.4.6, and threadpoolctl 3.6.0 (Moreau and maintainers 2025). NumPy supplies the dense arrays, eigensolvers, random generator, and linear-algebra operations used by the replay (Harris et al. 2020). Before importing NumPy, the launcher overwrites `MKL_NUM_THREADS`, `NUMEXPR_NUM_THREADS`, `OMP_NUM_THREADS`, and `OPENBLAS_NUM_THREADS` with 1.

Generated schema 1.1 records a path-free runtime fingerprint:

- Python implementation, version, compiler, build, word size, and byte order;
- operating-system release, version, and machine architecture;
- exact package versions;
- NumPy C/C++ compiler metadata, host / build machine metadata, BLAS / LAPACK identity, and SIMD baseline / found sets;
- runtime thread-pool API, implementation, version, architecture, threading layer, and observed thread count.

Hostnames, executable paths, environment paths, and wheel build paths are omitted so that two equivalent environments need not differ only because they reside in different directories. The clean replay identified CPython 3.12.10, MSC v.1943, 64-bit AMD64 Windows 11 build 10.0.26200, NumPy 2.4.6, and OpenBLAS 0.3.31.188.0 with pthreads, SkylakeX dispatch, and one runtime thread.

Version pins do not lock an operating-system image, CPU microcode, Python binary hash, NumPy wheel hash, BLAS binary, or all transitive native behavior. Therefore cross-platform byte identity is not claimed. Byte identity is a same-runtime acceptance check, while numerical agreement under a different backend would require tolerance-based review.

4.2 Disposable Clean-Environment Reproduction

The Draft 5 verification pass created a new virtual environment under the Windows temporary directory using the exact CPython 3.12.10 interpreter. Only the two pinned requirements were installed. The environment then generated the canonical result and a second result, ran the complete unit suite, and passed dependency compatibility checks. The two result files were byte-identical with SHA-256 9f48dadbe876dc29ae8cc64c646b49641d3ce81024f743098adf6bb4ceebfd1. All 19 executable replay and certificate tests passed in that disposable environment. The current package contains 22 tests after adding three theorem-source identity and anchor checks; those later checks were rerun against the current private sources and do not alter the numerical result.

The full interpreter, package-binary, source, requirement, and output hashes are recorded in reviews/CLEAN-ENVIRONMENT-REPRODUCTION.md. The temporary environment is not part of the package and is deleted after final package, manifest, and stage checks. This is a genuine fresh local environment build, but it is not independent because it uses the same host, operator, source tree, and investigation.

4.3 Finite Regression Levels and Tolerances

For $N=8, 16, 32, 64$, the replay constructs the closed-interval Farey sequence, builds the dense weighted Laplacian in deterministic vertex order, computes a Hermitian eigendecomposition, and checks:

1. one-dimensional kernel;
2. normalized constant-vector residual;
3. commutator orthogonality to the kernel;
4. Mertens trace identity;
5. per-vertex weighted-degree bound;
6. global operator-norm bound;
7. Commutator-Pythagoras residual for all four pseudoinverse cutoffs.

For each r_{cond} , eigenvalues above $r_{\text{cond}} * ||L_N||$ are inverted. Reporting all four rows exposes sensitivity to the numerical pseudoinverse threshold. No cutoff is selected after inspecting a target result.

4.4 Fixed-Seed Operator Baseline

At level 64, each Farey edge receives a pseudorandom phase from NumPy's default generator with seed 20260612. No potential, phase, threshold, loss, or target parameter is fitted. Eigenvalues above $1e-10$ are retained, and the 30th through 95th percentile window defines the bulk. Nearest-neighbor spacings are normalized by their mean and compared with GUE Wigner, GOE Wigner, and Poisson spacing CDFs using the two-sided one-sample Kolmogorov-Smirnov distance. For ordered spacings x_i , the implementation takes the maximum of $i/n - F(x_i)$ and $F(x_i) - (i-1)/n$, so both empirical-CDF limits are included. The spectrum is not unfolded beyond global mean-spacing normalization; these distances are therefore descriptive diagnostics, not universality tests.

The zeta-zero spacing literature motivates why such controls are scientifically interesting (Odlyzko 1987), but the present baseline never reads a zeta-zero table. One seed and one level cannot establish a limiting distribution, universality class, or relation to zeta zeros.

4.5 Permanent Zero-Data Omission

The private zero table used by the predecessor is not included. Draft 5 makes a final scope decision: the zero-data comparison is PERMANENTLY-OMITTED-FROM-THIS-REVISION. The generated object has null source / license / checksum fields, no parser test, no external datasets, and an empty result array. Historical zero-data metrics are not quoted as evidence.

This closes the ambiguity that existed when omission was described as a temporary release gate. The Paper 8 revision no longer depends on acquiring or licensing zero data. A later zero-data study, if separately approved, must be a distinct provenance-complete research object with its own source, license, checksum, extraction record, parser, analysis plan, and review. Permanent omission from this revision is not a null statistical result and does not grant permission to use the private table.

4.6 Exact-Scope Manifest

build_manifest.py hashes every regular package file in canonical relative path order. It excludes only MANIFEST.sha256 itself and interpreter caches under __pycache__ or with suffix .pyc. Symbolic links are rejected rather than followed, preventing an apparently internal manifest row from silently hashing content outside the package.

verify_manifest.py recomputes live scope and rejects unlisted files, out-of-scope rows, duplicates, noncanonical order, malformed digests, absolute or traversal paths, missing files, and hash mismatches. Manifest verification therefore checks package inventory completeness within the declared scope, not only the hashes of rows already listed.

4.7 Reproducibility ladder and release threshold

The manuscript uses a five-level reproducibility ladder:

1. **Internal deterministic replay:** current source regenerates the expected bounded output.
2. **Fresh same-host reconstruction:** a disposable environment rebuilds and reproduces the result on the same host.
3. **Independent native-stack reproduction:** another investigator reproduces the result on a materially different operating system, architecture, or numerical backend using declared tolerances where byte identity is inappropriate.
4. **Independent mathematical and statistical review:** specialists audit the theorem dependencies, numerical methods, and descriptive spectral analysis.
5. **Release-object verification:** the author-approved manuscript, code, notices, source locators, metadata, and exact provider files are rendered, hashed, and read back.

This package has completed levels 1 and 2 only. The Version 2 package organizes levels 3 through 5 as explicit gates rather than treating them as implied by local reproducibility.

5. Results

5.1 Finite Implementation Regressions

N	vertices	L _N	kernel dimension	max absolute identity residual	status
8	23	2.535387080015	1	3.6e-15	pass
16	81	2.592317693318	1	4.34e-14	pass
32	325	2.622073653381	1	1.237e-13	pass
64	1261	2.637294915958	1	2.0641e-12	pass

At all four levels, the kernel, commutator orthogonality, Mertens trace, weighted-degree bound, operator-norm bound, and all cutoff rows pass their declared tolerances in the recorded runtime. The field name `tier_1_theorem_regression` is retained for continuity, but the rows are implementation regressions and do not prove the external statements.

5.2 Operator-Only Descriptive Baseline

Quantity	Generated value
Farey level	64
seed	20260612
bulk modes	820
nearest-neighbor spacing samples	819
KS vs GUE Wigner surmise	0.727395592237
KS vs GOE Wigner surmise	0.698533884657
KS vs Poisson	0.577028573553
rounded-eigenvalue SHA-256 prefix	308fe8684567ce7b

The configuration is far from both Wigner surmises under the specified finite comparison. No uncertainty interval, seed ensemble, level-scaling analysis, or target-data comparison is available. The result cannot support a limiting law, target-specific fit, or RH claim.

5.3 Deficit Diagnostics

At levels 8, 16, 32, 64, 96, 128, the exact finite Mertens-ratio formula is converted to float64 and evaluated against the working envelope $\epsilon=0.25$, $c=1$. The status is NO-BOUNDED-FLAG. The positive-deficit log-log slope is -3.628419766308 , but the evaluated tail is not wholly below the floor. This is not an asymptotic pass, and $c=1$ is not an effective theorem constant.

A synthetic N^{-6} fixture at levels 16, 32, 64, 128, 256, 512 produces BOUNDED-DIAGNOSTIC, with slope -6 . It has no coverage premise and no evidentiary use. A separate unit fixture supplies schema-complete self-attested coverage metadata. Even there, the status is CONDITIONAL-INCONSISTENCY, external review is required, and theorem authorization remains false.

5.4 Omitted and Unimplemented Rows

The zero-data comparison is permanently omitted from this revision and has no result rows. Holdout execution, fake-target execution, budget equivalence, and the engine all-runs ledger are UNIMPLEMENTED. The trained-engine result array is empty. The Tier 3 publication gate is closed.

6. Tier 3 Protocol Boundary

Reproducibility problems in computational research are not repaired by a seed alone; protocols, budgets, run selection, and negative outcomes also matter (Henderson et al. 2018). Paper 8 specifies these future controls so that current absence cannot be mistaken for implementation.

6.1 Holdout Control

A future admissible design must train on zeros $1..K$, freeze every trainable parameter and preprocessing statistic, evaluate on zeros $K+1..2K$, and evaluate on a disjoint high window. Train, adjacent-holdout,

and high-window metrics must be reported together. No executable split, freeze enforcement, or protocol-compliant result exists here.

6.2 Fake-Target and Budget Controls

Future controls must include GUE-sampled, Poisson-sampled, and shuffled target sequences under identical curricula, budgets, seeds, preprocessing, stopping rules, and reporting. A target switch alone is insufficient; a budget-equivalence validator and complete outcome accounting are required. No such implementation or result exists in this package.

6.3 All-Runs Policy

For future Tier 3 work, a run begins when a process starts model execution after configuration validation. Before its first update, it must receive a unique run identifier and immutable configuration hash. The append-only ledger must include completed, failed, interrupted, aborted, and null runs.

Each row must record parent / restart identity, code and configuration hashes, input-data identity and checksum, environment identity, seed, budget, stopping rule, start and terminal state, terminal reason, metric trajectory or its absence, final parameters or their absence, spectra or their absence, and hashes of artifacts that exist. A recovery is a linked new run, not a replacement. Selective deletion, renumbering, or omission after inspecting an outcome is prohibited.

This is a policy specification only. The package has no append-only storage, run-launcher integration, crash capture, budget validator, training engine, or ledger tests. Its machine-readable status remains SPECIFIED-NOT-IMPLEMENTED.

7. Provenance, Citations, and Licensing

reproducibility/PROVENANCE.md identifies the historical PDF, review dossier, predecessor verifier and certificate, evidence plan, mathematical sources, and predecessor scripts by public-safe filename and SHA-256. Source texts and private engine-side files are not redistributed. The staged code is a standalone reimplement and includes no private run log, private configuration, credential, provider token, correspondence, zero table, or machine-local source path.

The citation audit distinguishes primary research records from background references. The historical DOI and companion manuscripts own program-specific claims. Hardy, Ingham, and Odlyzko are cited only for primary mathematical and computational context. The NumPy paper supports the software identity and role of the array library, not native-binary determinism. Henderson et al. supports the need for controlled reporting in computational work, not any result in this package. No source is used to transfer support from a related paper into an unimplemented Paper 8 result.

reproducibility/LICENSES.md is a disposition record, not a license. The predecessor's CC BY 4.0 statement motivates a proposal for the revised text, but the author must confirm rights, approve the exact notice, and authorize publication. No LICENSE file is present. New code remains private with no public license grant. MIT or Apache-2.0 are possible future choices, not grants. Generated JSON has no separate public license. No external data is included.

8. Limitations and Open Gates

1. The package does not independently contain or verify the external theorem proofs; numerical agreement is regression evidence only.
2. The clean replay uses a new environment but the same host and investigator; independent reproduction remains open.
3. Python and package versions are pinned, but operating-system image, CPU, interpreter binary, wheel, and BLAS binary are not locked by the environment files.
4. Cross-platform byte identity is not claimed.
5. The operator baseline uses one level and one seed; ensemble and scaling studies are absent, and no local spectral unfolding is performed.
6. The zero-data analysis is permanently outside this revision.
7. The working $c=1$ envelope is not an effective theorem constant.
8. No real exact-set floor-attainment coverage premise is supplied.
9. Float64 comparisons are diagnostic and do not adjudicate exact inequalities.
10. Holdout, fake-target, budget-equivalence, and all-runs controls are unimplemented.
11. No trained-engine evidence exists.
12. Independent mathematical, numerical, and statistical review remain open.
13. Text is packaged under CC BY 4.0 and executable code under MIT.
14. Version 2 is locally validated; independent mathematical, numerical, statistical, and cross-platform review remains recommended.
15. No correction notice, revised deposit, repository push, mirror mutation, tagged release, venue submission, or other repository release has been performed by this package.

9. Conclusion

This Version 2 successor replaces a promised historical reproducibility object with a bounded artifact whose claims, omissions, environment, and inventory are inspectable. The finite weighted Farey-Laplacian checks and fixed-seed operator baseline replay under the exact recorded CPython, NumPy, and OpenBLAS runtime. A disposable second environment produced byte-identical JSON and passed all tests. That closes a local clean-environment construction gate, but not independent or cross-platform reproduction.

The central correction is logical and executable. An infinitely-often floor does not govern every finite set. Exact-set coverage is a separate proof obligation, and self-attested JSON cannot discharge it. The package therefore permits a conditional inconsistency warning but never a machine-authorized theorem contradiction. This prevents a finite slope, arbitrary working constant, or metadata label from being promoted into proof language.

The revision also resolves the zero-data ambiguity by permanently removing that comparison from this paper. The retained evidence is narrower: finite regressions, one descriptive operator control, and synthetic semantics tests. Tier 3 remains a protocol specification with no implementation and no result.

Nothing in this artifact proves RH or physical SIT. Its contribution is a stricter reproducibility object in which theorem ownership, numerical regression, descriptive control, synthetic fixture, permanent omission, and unimplemented protocol remain separate in source, tests, generated output, manuscript language, and release gates.

Data and Code Availability

This is the locally validated Version 2 successor for Zenodo record 21231891. Its release package contains source, tests, version specifications, generated output, provenance and review documents, and an exact-scope manifest. It contains no external dataset and no training engine. The clean virtual environment used for Draft 5 verification is disposable and is not part of the package.

The exact package is prepared for a new Zenodo version under concept DOI 10.5281/zenodo.21231890, followed by OSF parity verification. repository release remains a separate coordinated portfolio action and has not yet occurred.

Version Lineage and Source-Faithful Correction

Public Version 1 remains preserved at DOI 10.5281/zenodo.21231891. Version 2 does not overwrite or backdate that record. It retains the original Paper 8 goal of making the H_SIT computational claims inspectable while correcting the parts that the deposited PDF could not support.

The successor:

- replaces the historical one-command claim with an actual version-pinned finite replay;
- separates proof-owned statements from regression checks;
- replaces an automatic theorem-verdict rule with coverage-safe conditional semantics;
- permanently omits the unprovenanced zero-data comparison;
- keeps holdout, fake-target, budget, and all-runs controls visibly unimplemented;
- records a fresh same-host reconstruction without calling it independent or cross-platform replication;
- preserves the finite model, operator baseline, Tier 3 protocol boundary, limitations, provenance, and bibliography.

The correction is additive to the legitimate research program. It does not erase the original motivation, turn a numerical regression into a proof, or assign later wording an earlier date.

Authorial Provenance

The underlying Super Information Theory research program and the conceptual work from which this paper descends were developed by Micah Blumberg across 2011-2026, with major foundations established before the public release of ChatGPT in November 2022. AI did not originate the underlying theory, hypotheses, or research program. During later manuscript and artifact development, AI-assisted tools were used under Blumberg's direction for bounded tasks including code formulation, mathematical checks, source organization, adversarial review, and editorial correction. Every scientific claim, interpretation, and release decision remains the author's responsibility and requires the review gates stated in this manuscript.

References

The machine-readable bibliography is `references.bib`. Citation keys used in this Version 2 manuscript are `blumberg2026paper8`, `blumberg2026weightedfarey`, `blumberg2026saturation`, `hardy1914`, `ingham1942`, `odlyzko1987`, `henderson2018`, `harris2020`, and `threadpoolctl360`.

References

- Blumberg, Micah. 2026a. *Reproducible Computational Evidence for the H_{SIT} Program*. Zenodo. doi:10.5281/zenodo.21231891.
- Blumberg, Micah. 2026b. *RH as a Saturation Law*. Zenodo. doi:10.5281/zenodo.21231877.
- Blumberg, Micah. 2026c. *The Weighted Farey Laplacian and the HSIT–Riemann Program*. Zenodo. doi:10.5281/zenodo.21231873.
- Hardy, G. H. 1914. “Sur Les Zeros de La Fonction Zeta(s) de Riemann.” *Comptes Rendus de l’Academie Des Sciences de Paris* 158: 1012–14.
- Harris, Charles R. et al. 2020. “Array Programming with NumPy.” *Nature* 585: 357–62. doi:10.1038/s41586-020-2649-2.
- Henderson, Peter, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. “Deep Reinforcement Learning That Matters.” *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, 3207–14. doi:10.1609/aaai.v32i1.11694.
- Ingham, A. E. 1942. “On Two Conjectures in the Theory of Numbers.” *American Journal of Mathematics* 64 (1): 313–19. doi:10.2307/2371697.
- Moreau, Thomas, and threadpoolctl maintainers. 2025. *Threadpoolctl: Thread-Pool Controls for Python*. Python Package Index. <https://pypi.org/project/threadpoolctl/3.6.0/>.
- Odlyzko, A. M. 1987. “On the Distribution of Spacings Between Zeros of the Zeta Function.” *Mathematics of Computation* 48 (177): 273–308. doi:10.1090/S0025-5718-1987-0866115-0.